

ASSCG: Just-Right Gating over Chattering for Fast-Slow LLM Planning in Autonomous Driving

Sining Ang^{1,2}, Yuan Chen³, Liu Haiyan⁴, Xuanyao Mao⁴, Jason Bao⁴,
Xuliang⁴, Bingchuan Sun^{4*}, and Yan Wang^{1*}

¹ Institute for AI Industry Research (AIR), Tsinghua University
angsn@mail.ustc.edu.cn, wangyan@air.tsinghua.edu.cn

² Department of Automation, University of Science and Technology of China

³ Beijing University of Aeronautics and Astronautics
chenyuan1@buaa.edu.cn

⁴ Lenovo Group Limited
{liuhy46,maoxy6,jbao,xuliang18,sunbc1}@lenovo.com

Abstract. Large language models (LLMs) can improve autonomous driving planning but are costly to query online, and existing fast-slow planners often rely on hand-designed triggering rules that either over-call the slow system or call it at the wrong times. We formulate slow-system invocation as a resource-aware sequential decision problem and propose the Adaptive Slow-System Control Gate (ASSCG), which makes frame-level Query/Cache/Drop decisions to refresh, reuse, or suppress slow guidance. ASSCG uses an RWKV backbone for efficient long-horizon gating and is trained with supervised fine-tuning followed by GRPO-style compute-aware reinforcement fine-tuning. We apply ASSCG to two different fast-slow architectures: (i) AsyncDriver on nuPlan Hard20 closed-loop evaluation, where ASSCG improves score to 67.28 (+2.28) while reducing average end-to-end inference latency by $\sim 60\%$; and (ii) a RecogDrive-based dual system that we build by replacing its original VLM-2B module with a lightweight ViT-based fast planner and adding an LLM slow planner, evaluated on NAVSIM, where ASSCG achieves 91.4 PDMS (+0.6) and increases average speed by $\sim 25\%$. The project page, including video visualizations and additional results, is available at <https://williamxuanyu.github.io/asscg/>.

Keywords: Autonomous driving · LLM · Dual system

1 Introduction

Motion planning is a core component of autonomous driving. Despite notable progress in learning-based planning and standardized benchmarks [11, 40], modern planners still struggle in complex, dynamic, long-tail interactions [5, 6, 12, 14]. In parallel, large language models (LLMs) exhibit strong commonsense reasoning and cross-domain transfer; with instruction tuning, they can internalize traffic

* Corresponding authors.

rules and scenario priors to provide more context-aware guidance [1, 2, 8, 10, 13]. This has motivated LLM-augmented planning.

However, LLM inference latency and compute cost make per-step deployment difficult. A practical compromise is the decoupled fast-slow paradigm: a real-time fast planner runs every frame while an LLM-based slow module provides high-level guidance at a lower frequency [3, 29, 30]. Yet in practice, peak performance often requires querying the slow module nearly every frame; reducing the query rate typically degrades results. More importantly, adding a slow LLM module does not always help and can even hurt in certain segments (Fig. 2), raising a central question: *when should the system rely on the fast planner, and when should it consult (or ignore) the slow module, to maximize closed-loop performance under a compute budget?*

Two coordination strategies are widely used (Fig. 1): (i) fixed-interval triggering and (ii) heuristic difficulty-based triggering. Fixed schedules ignore intra-scene variability, leading to over-calling in easy segments and under-calling in hard ones. Difficulty-based gating requires defining and estimating “scene complexity”, yet such proxies do not necessarily align with the *marginal utility* of invoking the slow module at a particular time. Empirically, VLMPanner evaluates multiple gating variants (rule-based and learned complexity criteria) across different settings and hyperparameters, but none improves over the always-querying baseline in terms of final score; the best accuracy is still achieved by per-frame slow-module invocation, while the best efficiency-accuracy trade-off comes from a learned complexity gate that slightly reduces score but significantly improves throughput [29]. Moreover, most existing schemes operate at the scene level with fixed fast/slow frequencies per class, overlooking fine-grained temporal dynamics within a single episode.

We propose the **Adaptive Slow-System Control Gate (ASSCG)**, an architecture-agnostic control interface that adaptively schedules and regulates a slow module at the frame level. Here, architecture-agnostic means that ASSCG only assumes a fast branch, an optional slow branch, and a cached guidance representation; the gate is still trained or lightly adapted for the target planner and benchmark. We formulate slow-module control as a sequential decision problem with query costs under partial observability (POMDP/SMDP), and recast it as sequence

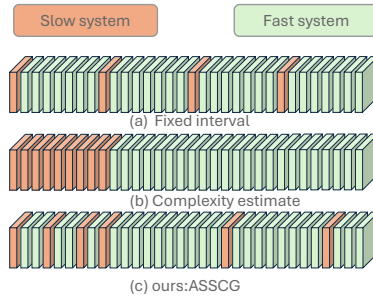


Fig. 1: Common fast-slow coordination strategies: (a) fixed-interval triggering ignores temporal variation, wasting compute in easy periods and missing critical moments; (b) difficulty/complexity-based triggering relies on imperfect proxies that not align with the value of slow reasoning, leading to unnecessary oscillation and mistimed queries; (c) ours (ASSCG), a learned gate that makes frame-level QUERY/CACHE/DROP decisions to balance planning quality and resource cost.

generation: the controller predicts a gating token per frame. This LLM-style formulation enables standard SFT-to-RL fine-tuning. Concretely, we implement ASSCG with an RWKV backbone [23] and fine-tune it with a compute-aware GRPO-style algorithm [27]. On nuPlan Hard20 closed-loop evaluation, integrating ASSCG into AsyncDriver (hereafter referred to as *AdaptiveAsyncDriver*) reduces average end-to-end latency by about 60% while improving the score by +2.28 (to 67.28). We further demonstrate the applicability of the same gating principle on a separate RecogDrive-based fast-slow instantiation evaluated on NAVSIM, where it improves PDMS by +0.6 and increases average speed by $\sim 25\%$. Our contributions are:

1. We analyze fast-slow collaboration and introduce *effective* and *failure* intervals to characterize when slow guidance helps or hurts over time.
2. We present ASSCG, a frame-level gate trained with GRPO-style reinforcement fine-tuning to optimize closed-loop metrics under query costs.
3. We evaluate ASSCG on two representative fast-slow instantiations and benchmarks: AsyncDriver on nuPlan closed-loop Hard20 and a RecogDrive-based fast-slow planner on NAVSIM, demonstrating improved performance-efficiency trade-offs across architectures and evaluation protocols.

2 Related Work

2.1 Language-augmented motion planning with LLMs

Large language models (LLMs) exhibit strong generalization and reasoning after pre-training on massive corpora, motivating their use in autonomous driving decision-making. Recent works encode ego state, surrounding agents, and map context as linguistic descriptions or BEV-style representations for LLM/VLM-based scene understanding and trajectory guidance [28, 36, 42]. For example, PlanAgent [42] structures scenes in BEV to guide a base planner, and LLM-ASSIST [28] integrates LLM reasoning with rule-based modules.

Purely language-mediated interfaces, however, face an expressivity-latency mismatch for time-critical control: limited context length and the difficulty of encoding precise continuous states can reduce reliability. Multimodal approaches—e.g., DrivingWithLLM [2], DriveGPT4 [35], and RAGDriver [38]—align vectorized or image/video inputs with language to enrich interpretation, but translating language outputs into stable closed-loop control remains challenging. Another line couples LLMs more tightly with low-level controllers for closed-loop driving, such as LMDrive [26] and LanguageMPC [33], yet these designs still rely on frequent LLM inference or serial decoding, stressing real-time responsiveness.

2.2 Fast-slow (dual-system) planners and adaptive LLM invocation

To better satisfy real-time constraints while leveraging LLM reasoning, fast-slow planners decouple a real-time fast planner from an LLM-based slow module that provides high-level guidance at a lower frequency [3, 30]. AsyncDriver [3] caches

slow guidance and reuses it across frames, amortizing cost but typically requiring careful scheduling to avoid stale or mis-timed interventions under non-stationary traffic.

A number of works study *when* to invoke the slow module. VLMPlanner [29] introduces CAI-Gate to adjust query frequency using simple/complex judgments from a VLM; its reported gains are marginal under multiple gating criteria, suggesting that generic scene-difficulty proxies may not reliably track the *value* of invoking the slow planner. Uncertainty-based triggering (e.g., FASIONAD [24]) calls the slow module under predicted waypoint uncertainty, but uncertainty calibration in interactive long-tail scenes can be brittle. Task-specific systems (e.g., Chameleon [41]) allocate extra slow reasoning for detected edge cases, often requiring curated traces or additional annotations.

Recent concurrent work. AdaDrive [39] proposes an adaptive slow-fast framework that learns *when* to activate LLM assistance via an adaptive activation loss, and further introduces a continuous fusion mechanism that scales the LLM contribution based on scene complexity and prediction confidence. Our work is complementary but differs in several aspects: (i) we learn an *architecture-agnostic, frame-level* discrete gate with explicit QUERY, CACHE, and DROP actions, where DROP enables intentionally suppressing slow guidance when it is likely to be harmful; (ii) we train the gate with supervised pretraining followed by compute-aware RL fine-tuning to directly optimize task-level driving metrics under an explicit query-cost constraint; and (iii) rather than scaling LLM influence using confidence as a primary signal, we focus on *when to consult* and *whether to trust* the slow module—in our setting, the fast planner already performs probability-based trajectory aggregation (GameFormer [16]), which can make confidence among shortlisted candidates less discriminative for further modulation. Overall, our goal is to learn just-right *timing* and *usage* of slow reasoning under resource constraints.

3 Analysis

3.1 Setup and assumptions

Following the AsyncDriver paradigm, we treat the LLM-based slow system as a high-level intent generator that does not need to be queried at every frame. The real-time fast planner consumes and caches the most recent slow-system guidance and continues planning until the next refresh. Concretely, let q_k denote the frames at which the slow system is invoked, z_{q_k} the resulting high-level guidance (e.g., latent features, waypoints, or policy logits), and u_t the low-level control produced by the fast planner at frame t using the latest cached guidance z_{q_k} , where $q_k \leq t < q_{k+1}$.

3.2 Concepts and operational definitions

To analyze when and how the slow system should be queried, we introduce three intervals and provide operational criteria for measurement in practice.

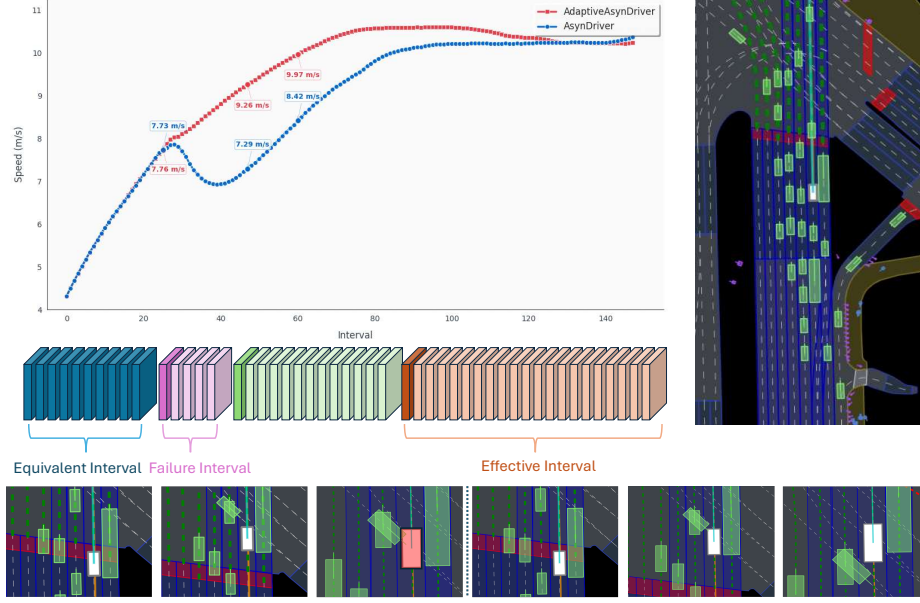


Fig. 2: Straight-driving case study comparing AsyncDriver (always querying the slow planner) and AdaptiveAsyncDriver with ASSCG (querying only at frames 0, 22, and 89). Despite far fewer slow-system calls, ASSCG avoids a collision and achieves better closed-loop behavior. Frames 0–25 form an Equivalent Interval (EI); frames 25–40 reveal a Failure Interval (FI) where slow guidance is harmful; and frames 1–25 and 90–149 are Effective Intervals (EfI) for guidance issued at frames 0 and 89, respectively. Additional qualitative visualizations are provided in the supplementary material.

Equivalent Interval (EI). A post-query time window in which re-querying the slow system would be redundant because the incremental information is negligible. Operationally, for $t \in [q_k, q_{k+1}]$: (i) $\text{similarity}(z_{q_k}, z_{t'}) \geq \tau_{sim}$ for all t' in the window, where similarity can be cosine similarity in the slow-system latent space or an embedding distance after a projection; or (ii) the induced control difference is negligible, e.g., $\|u_t(z_{q_k}) - u_t(z_{q_{k'}})\| \leq \tau_{ctrl}$ for a hypothetical re-query at frame $q_{k'} = t$. Intuitively, EI indicates that the slow guidance remains valid and “fresh,” and repeated slow queries would not change the planned motion in a meaningful way.

Effective Interval (EfI). The span during which the current slow guidance is actually used by the fast planner before being refreshed, i.e., $[q_k, q_{k+1}]$. By definition EfI is the cache-validity window; in practice it may be longer or shorter than EI. When $EfI \approx EI$, caching is efficient; when $EfI \gg EI$, stale guidance can accumulate; when $EfI \ll EI$, compute is wasted.

Failure Interval (FI). A time window in which invoking the slow system degrades downstream performance relative to not invoking it. Operationally, for a candidate call at t : $\Delta J_t = J(u_t \mid \text{call at } t) - J(u_t \mid \text{no call at } t) < -\tau_{perf}$, where J

aggregates safety and efficiency objectives (e.g., collision, lane-keeping, progress, comfort). Typical causes include misestimated distances, occlusion-induced hallucinations, or misinterpretation of local interaction rules. FI emphasizes that more frequent slow reasoning is not always better.

These definitions align with a value-of-information perspective: query the slow system only when the expected improvement in J exceeds the query cost and the risk of FI.

3.3 Case study and empirical observations

Fig. 2 illustrates a straight-driving episode comparing AsyncDriver (querying the slow module every frame) and our approach (querying only at frames 0, 22, and 89). Despite making only three slow-module calls, our gate avoids a collision and outperforms the always-query baseline. Notably, frames 0–25 behave as an Equivalent Interval (re-querying yields negligible change), while frames 25–40 expose a Failure Interval where slow guidance is detrimental in this instance. This example motivates the need for a gate that can both *reuse* and *suppress* slow guidance depending on context.

Beyond a single case: fixed-schedule grid search across scenario types. To check whether the above behavior is isolated, we run a grid search of *fixed* slow-module query schedules on nuPlan Hard20, separately for each of the 14 scenario types. We evaluate fixed query stride $K \in \{1, 3, 5, 10, 20, 50, 100\}$ (query every K frames) and a *Never-Query* setting that disables the slow module. Across types, the best fixed schedule varies substantially—from always querying ($K=1$) to rarely querying ($K=50$), and in one case *Never-Query* performs best. Notably, this grid search assigns a *single* stride to all episodes within a type; the optimal stride can still differ across episodes within the same type due to intra-scene temporal dynamics. This further motivates learning a frame-level policy that adapts query decisions online. Detailed per-type results are provided in Appendix A.

3.4 Implications for gating policy design

The analysis motivates a frame-level gating policy with three objectives:

1. Minimize redundant slow queries within Equivalent Intervals by reusing previously issued slow guidance and deferring new slow calls while the scene and control remain within the same equivalence window.
2. Suppress slow queries in segments unsuitable for LLM involvement (e.g., Failure Intervals), thereby avoiding adverse interventions.
3. Extend Effective Intervals by continuing to use cached slow guidance until a refresh is needed, subject to a maximum reuse horizon.

ASSCG operationalizes these objectives by casting the slow-trigger decision as a sequential problem under a constrained query budget. It seeks to maximize J by invoking the slow system only when the expected marginal benefit justifies the query cost; otherwise, it reuses cached guidance and leverages the fast planner. Implementation details are provided in the following section.

4 Methodology

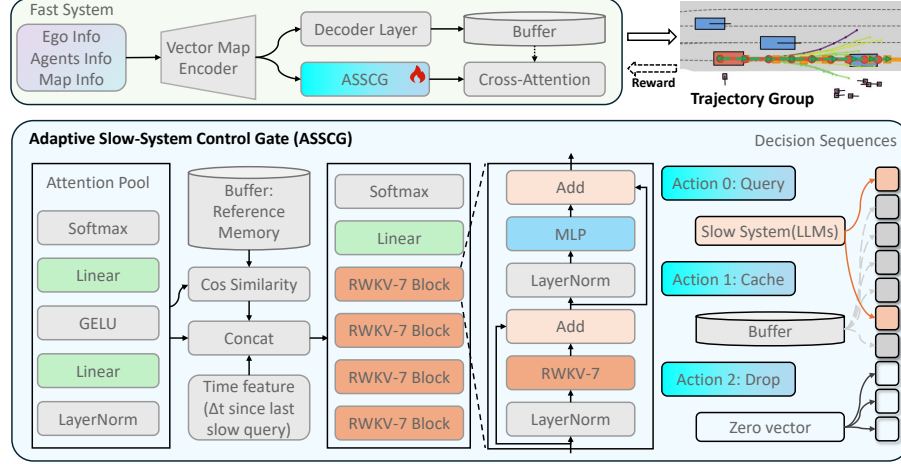


Fig. 3: Overview of our framework. We couple a GameFormer-based [16] fast planning system with an LLM-based slow system, coordinated by an Adaptive Slow-System Control Gate (ASSCG). At each frame, encoded vector-map (and ego/agent features) is fed to both the fast decoder and ASSCG. ASSCG outputs a discrete gating action: **Query** invokes the slow system to refresh a reference-memory buffer, **Cache** reuses the buffer without querying the slow system, and **Drop** ignores the buffer (zero vector) to prevent stale memory from affecting planning. Conditioned on the gating decisions, the fast decoder fuses fast features with the (optional) buffered reference via cross-attention and outputs a trajectory for one rollout. A *trajectory group* is obtained by sampling multiple rollouts, whose scores are converted to rewards to optimize the per-frame gating policy with group-based RL (GRPO). Inside ASSCG, a lightweight attention pool aggregates cues, which are compared with the buffer via cosine similarity and concatenated with time feature before an RWKV backbone predicts the gating policy.

4.1 Overview

Our framework follows a decoupled fast–slow architecture. A learning-based real-time planner (fast system) runs every frame, while an LLM-based planner (slow system) is invoked on demand to provide high-level guidance that is cached in a buffer and fused into the fast planner via cross-attention. The Adaptive Slow-System Control Gate (ASSCG) decides at each frame whether to (i) **QUERY** the slow system and refresh the buffer, (ii) **CACHE** and reuse the buffered guidance, or (iii) **DROP** the guidance by substituting a null feature.

Training pipeline and parameter freezing. Following AsyncDriver, the underlying fast–slow planner is trained in three stages: (i) we first train the fast planner (GameFormer-style) independently; (ii) we then fine-tune the slow LLM

(LLaMA2-13B) with LoRA for driving-domain adaptation; and (iii) we subsequently train the full fast-slow planner end-to-end while freezing the LLM weights, so that the fast planner and fusion modules adapt to the LLM guidance without updating the LLM itself. After inserting ASSCG, we perform an additional, separate training stage for the gate: we freeze *all* parameters of the original AsyncDriver and optimize only ASSCG. This isolates the effect of improved slow-system scheduling and selective usage from further planner adaptation.

4.2 Model architecture

The core dual-system architecture follows the AsyncDriver paradigm. Let m_t denote the vectorized map-and-agent features produced by a Vector Map Encoder at frame t ; the fast planner’s decoder consumes m_t to predict low-level control and a short-horizon trajectory. When available, cached slow guidance b_t from the buffer is injected into the decoder through a cross-attention path. The slow system (LLM) is queried only when gated “on,” producing high-level intent z_t (e.g., latent features, waypoints, or policy logits), which is written to the buffer.

Concretely, in a decoder block l , we extend the standard attention with an instruction-injection:

$$s^{l+1} = g \cdot \text{softmax} \left(\frac{QK_h^T}{\sqrt{C}} \right) V_h + \text{softmax} \left(\frac{QK_{s^l}^T}{\sqrt{C}} \right) V_{s^l}$$

where h is the instruction feature projected from the slow guidance (last-token hidden state after a small adapter), s^l is the scene feature at block l , and g is a learnable gate.

Per iteration, we obtain vectorized ego&agent histories and map elements in the ego-centric frame, similar to GameFormer-style encoders. The Vector Map Encoder produces tokenized features for lanes, topology, and dynamic agents. These tokens drive the fast planner, while ASSCG decides whether to reuse features in the buffer, refresh the buffer with new LLM-derived features, or bypass LLM features altogether.

4.3 Buffering and guidance representation

Buffer content. The buffer stores a compact slow-guidance representation $b = \text{Proj}(z)$, along with map tokens and the current timestamp (frame). **Read/write policy.** When the slow system is queried at frame t , we overwrite $b \leftarrow \text{Proj}(z_t)$. When not queried, we reuse the previous b . When the gate predicts “drop,” we expose a null vector b_0 to the planner, effectively disabling slow guidance.

4.4 ASSCG: Adaptive Slow-System Control Gate

The gating policy must determine how the planner should interact with the buffer at every frame— whether to refresh it with new slow guidance, keep using

the existing contents, or intentionally ignore it to avoid harmful interventions. This decision depends on the temporal context, cached features, and the learned scheduling strategy.

Problem framing: At each frame t , ASSCG produces an action $a_t \in \{0, 1, 2\}$:

- 0 = Query (invoke the slow system and refresh the buffer),
- 1 = Cache (reuse the buffer as-is),
- 2 = Drop (ignore the buffer by substituting a null feature).

This realizes our interval analysis: CACHE extends effective intervals, QUERY marks the start of a new interval, and DROP suppresses suspected failure intervals. The first frame of every episode is forced to QUERY to initialize the buffer, so CACHE is never applied without valid slow guidance.

Inputs to the gate: Current scene embedding x_t from the Vector Map Encoder (token sequence); Reference memory b_{ref} from the buffer (the last slow-guidance embedding); Cosine similarity $s = \cos(Pool(x_t), b_{ref})$, where $Pool(\cdot)$ is a lightweight attention pooling that summarizes x_t ; Time feature is Δt since the last slow query (normalized via logarithmic scaling). The gate concatenates $[Pool(x_t), s, \Delta t]$ as its input feature.

Attention pooling: We employ a compact attention-pooling block (Linear–GELU–Linear–Softmax with LayerNorm) to summarize the token sequence. It mirrors the Map Adapter design in AsyncDriver for map embeddings, ensuring architectural consistency and integrating vectorized map features with minimal computational overhead.

Why RWKV for gating. ASSCG must make low-latency, frame-level decisions while tracking long-horizon temporal context (up to an entire episode). We adopt RWKV [23] for two reasons. First, its recurrent time-mixing can be viewed as maintaining a compact, recursively updatable latent state, which matches the intuition that the gate benefits from an internal belief state under partial observability. Second, RWKV provides a favorable deployment profile for online decision-making with long context: after state caching, it has per-step linear time and constant memory, avoiding the quadratic attention cost of Transformers as the episode length grows. We empirically compare RWKV with a parameter-matched Transformer alternative in the ablation study (§6.4).

RWKV backbone and head: The pooled feature and auxiliaries are fed to a 4-layer RWKV-7 stack. A linear classification head with softmax outputs $\pi(a_t | s_t)$ over {QUERY, CACHE, DROP}. As discussed above, RWKV offers an efficient long-horizon gating backbone; quantitative accuracy/latency comparisons against Transformer-based gating are reported in the ablations.

5 Data Collection and Training Protocol

5.1 Datasets

Scenario types and splits. We follow the nuPlan Challenge 2023 closed-loop setting and evaluate on the 14 official scenario types [11]. For a fair comparison

with prior fast-slow planners, we use the *Hard20* test split released by Async-Driver [3] and VLM-planner [29] (same scene IDs and evaluation protocol), and report results on this fixed split throughout.

SFT training pool. From the training split, we randomly sample 60 scenes per scenario type (14 types). We collect training rollouts by executing a set of fixed slow-module query schedules in the nuPlan simulator, using query stride $K \in \{1, 3, 5, 10, 20, 50, 100\}$ (query every K frames) and a *Never-Query* setting that disables the slow module. For each rollout, we log per-frame encoder features, the slow-module output (when queried), the gate action token, and metric components of the nuPlan score (Eq. 13 in Appendix C).

To obtain supervision for the gate, we select, *for each scene*, the rollout with the highest closed-loop score among the above schedules, and use its per-frame QUERY/CACHE/DROP action sequence as pseudo labels for supervised fine-tuning (SFT).

RL data preparation. We construct an RL dataset by rolling out the SFT-initialized gate policy in the nuPlan simulator and then training from the logged trajectories. The gate action space has $A = 3$ actions (QUERY, CACHE, DROP); each episode spans the full closed-loop horizon.

We adopt an offline RL workflow: using the frozen SFT policy π_0 , each training scene is rolled out $R = 10$ times with different random seeds. Actions are selected via entropy-shaped temperature scaling with ε -mixing. For a state s , let $p = \text{softmax}(\text{logits}(s))$ over the action set A , and define

$$\begin{aligned} H(s) &= -\sum_{i=1}^A p_i \log p_i, & H_{\text{norm}}(s) &= \frac{H(s)}{\log A} \in [0, 1], \\ T(s) &= T_{\text{low}} + (T_{\text{high}} - T_{\text{low}})(H_{\text{norm}}(s))^\alpha, \\ \varepsilon(s) &= \varepsilon_{\text{low}} + (\varepsilon_{\text{high}} - \varepsilon_{\text{low}})(H_{\text{norm}}(s))^\alpha. \end{aligned} \tag{1}$$

With $p_T(a \mid s) = \text{softmax}(\text{logits}(s)/T(s))$ and $U(a) = 1/A$, the behavior distribution is

$$\begin{aligned} p_{\text{mix}}(a \mid s) &= (1 - \varepsilon(s))p_T(a \mid s) + \varepsilon(s)U(a), \\ a &\sim \text{Cat}(p_{\text{mix}}(\cdot \mid s)) \end{aligned} \tag{2}$$

using $T_{\text{low}}=0.9$, $T_{\text{high}}=1.6$, $\varepsilon_{\text{low}}=0.02$, $\varepsilon_{\text{high}}=0.12$, $\alpha=1.0$. We force a QUERY at the first step of each episode to initialize the buffer. The resulting static dataset D logs per-step s (encoder features), a , reward metrics for offline reward reconstruction (e.g., TTC, Lim, Comf, Prog; see in Appendix C), the behavior log-probability $\log b = \log p_{\text{mix}}(a \mid s)$, as well as $H_{\text{norm}}(s)$, $T(s)$, $\varepsilon(s)$, and meta-data (done, episode_id, step_idx, timestamps).

5.2 Training protocol

We train ASSCG in two stages: supervised fine-tuning (SFT) on pseudo labels, followed by compute-aware RL fine-tuning.

Stage I: Supervised fine-tuning. Over the action set $a_t \in \{\text{QUERY}, \text{CACHE}, \text{DROP}\}$, we train a 3-way classifier using the per-frame action sequence from the highest-scoring fixed-schedule rollout of each scene (§5.1). We minimize cross-entropy with class-balancing to avoid the degenerate CACHE-only policy. We use AdamW (lr 3×10^{-4} , weight decay 0.01) with cosine decay and 10% warmup, label smoothing 0.05, and gradient-norm clipping at 1.0.

Stage II: Compute-aware RL fine-tuning. The nuPlan closed-loop score (Eq. 13 in Appendix C) is computed as a weighted sum of per-episode metrics, normalized by the sum of weights $W = 16$, and multiplied by gating terms (e.g., collision/drivable/direct). Since Eq. 13 in Appendix C is episode-level, it provides sparse supervision. We introduce dense intermediate feedback via potential-based shaping while preserving alignment with the official weighting. Let $W_{\text{np}} = 11$ denote the total weight of the non-progress terms (TTC , Lim , $Comf$). Define

$$N_t = 5 \text{TTC}_t + 4 \text{Lim}_t + 2 \text{Comf}_t, \quad G_t = \text{Coll}_t \times \text{Drivable}_t \times \text{Direct}_t, \quad (3)$$

and the normalized potential

$$\Phi_t = \frac{1}{W} N_t G_t = \frac{W_{\text{np}}}{W} \cdot \underbrace{\left(\frac{N_t}{W_{\text{np}}} G_t \right)}_{\psi_t \in [0,1]}. \quad (4)$$

The per-step reward is

$$r_t = (\gamma \Phi_{t+1} - \Phi_t) - \lambda_{\text{call}} \cdot \mathbb{I}[a_t = \text{QUERY}], \quad (5)$$

with an additional terminal progress settlement

$$r_T \leftarrow r_T + \frac{W - W_{\text{np}}}{W} \text{Prog}_T G_T. \quad (6)$$

With $\gamma \approx 1$, the shaping term telescopes, yielding a constant-shifted version of the non-progress part of Eq. 13 in Appendix C, and the terminal term restores the progress component. This yields an objective consistent with the official evaluation but with denser learning signal. The compute-awareness is explicit in the objective: every QUERY action receives a penalty controlled by λ_{call} , while CACHE and DROP avoid this cost. Increasing λ_{call} shifts the policy toward longer effective query intervals, trading slow-module usage against closed-loop score in a controlled way.

We fine-tune the policy using a GRPO-style clipped policy-gradient objective with (i) off-policy correction using logged behavior probabilities, (ii) KL regularization to the SFT reference policy π_{ref} , and (iii) an entropy bonus for exploration. Concretely, with importance ratio $\rho_t = \pi_\theta(a_t|s_t)/b(a_t|s_t)$ truncated by ρ_{max} , we apply PPO-style clipping with range ϵ ; full objective and advantage estimation details are provided in the supplementary material.

Hyperparameters. Unless otherwise stated: $\gamma=0.99$, GAE $\lambda=0.95$, $\lambda_{\text{call}}=0.001$, clip $\epsilon=0.2$, $\rho_{\text{max}}=10$, entropy coefficient $\eta=0.01$, value loss coefficient $\lambda_V=0.5$, and KL coefficient β is set to 0.01. We use AdamW with lr 1×10^{-4} and gradient-norm clipping at 1.0.

6 Experiments

6.1 Experimental Setup

General settings. Unless otherwise stated, all reported scores and inference latencies are measured with single-GPU inference on an NVIDIA A30. To reduce variance from RL fine-tuning, we report the average over three runs with different random seeds.

nuPlan setup. We evaluate in the nuPlan closed-loop reactive setting: non-ego agents follow IDM-based planners and respond to the ego’s actions. The simulation runs at 10 Hz, and each planning iteration predicts an 8 s trajectory horizon. We use the official nuPlan closed-loop metrics; definitions and the composite score formula are summarized in Appendix C. For fairness with prior work, we use the fixed Hard20 split from AsyncDriver & VLMplanner (same scene IDs).

NAVSIM setup. We also evaluate on NAVSIM, a planning-oriented benchmark with a non-reactive (pseudo closed-loop) simulator. Each scene requires one-shot prediction of a 4 s future trajectory (8 waypoints at 0.5 s). We report the official PDMS metric (Appendix E).

6.2 Results on nuPlan (AsyncDriver-based Fast-Slow Planner)

On nuPlan Hard20, integrating ASSCG into AsyncDriver improves the overall score to 67.28 (+2.28 absolute) while reducing compute by lowering the slow-module query rate. Detailed per-scenario-type results are provided in Appendix D.1. Overall, we observe improvements on most scenario types, with the largest gains on lane-change and intersection-related categories, and only minor regressions on two types (type0 and type7), which are within a small margin.

In addition to the overall score, Table 1 reports a breakdown of nuPlan metric components. ASSCG improves drivable-area and driving-direction compliance as well as TTC, indicating that the gain is primarily driven by safer and more stable closed-loop behavior.

Beyond accuracy, ASSCG yields consistent efficiency benefits. Table 3 compares three settings enabled by the asynchronous fast-slow design of AsyncDriver: always querying the slow module, a fixed-stride schedule that queries once every 5 frames, and ASSCG. The 5-frame fixed schedule has nearly the same average latency as ASSCG (0.32 s/frame), providing a latency-matched baseline. Under this comparable efficiency budget, ASSCG improves the score from 64.27 to 67.28 (+3.01), indicating that the gain comes primarily from *better timing and selective suppression* of slow-module calls rather than merely

Table 1: nuPlan Hard20 closed-loop evaluation: score breakdown. We report the overall score and the average values of metric components used by the official nuPlan score (Eq. 13 in Appendix C). *Drivable*: drivable-area compliance; *Direct*: driving-direction compliance; *Comf*: comfort; *Prog*: route progress; *Coll*: no at-fault collision factor; *Lim*: speed-limit compliance; *TTC*: time-to-collision within bound.

Method	Score	Drivable	Direct.	Comf.	Prog.	Coll.	Lim.	TTC
UrbanDriver [25]	35.35	75.53	97.12	98.56	85.23	55.21	81.62	47.84
GCPGP [12]	36.85	81.29	98.20	77.33	46.96	72.30	97.92	68.34
IDM [31]	53.07	84.94	98.02	83.15	64.79	74.01	96.38	60.57
GameFormer [16]	62.05	93.54	98.74	83.15	66.27	86.02	98.19	<u>74.55</u>
PDM-Hybrid [9]	64.05	95.34	99.10	75.98	67.93	87.81	99.57	72.75
PDM-Closed [9]	64.18	<u>95.69</u>	<u>99.10</u>	77.06	68.20	87.81	99.57	73.47
VLM-planner [29]	<u>66.66</u>	90.91	95.65	84.58	<u>84.29</u>	76.88	<u>99.37</u>	70.36
AsyncDriver	65.00	94.62	98.75	83.15	67.13	85.13	98.15	73.48
<i>ours</i>	67.28	96.40	99.27	<u>86.66</u>	68.16	<u>86.99</u>	98.14	75.93

reducing their frequency. On the Hard20 test split, ASSCG achieves an average effective query interval of 5.26 frames, corresponding to a slow-query rate of roughly 19.0% of frames. This is close to the fixed 5-frame schedule in latency, but differs in timing and in whether cached slow guidance should be trusted or dropped.

Compared with VLM-planner, ASSCG has lower route progress but improves the safety-related and rule-compliance terms (drivable area, driving direction, comfort, collision, and TTC), yielding a higher composite score. This suggests that VLM-planner is more aggressive on Hard20, whereas ASSCG trades some progress for more stable closed-loop behavior.

6.3 Results on NAVSIM (RecogDrive-based Fast-Slow Planner)

We evaluate ASSCG on NAVSIM using a RecogDrive-based fast-slow instantiation (Appendix F.3). We keep the original VLM module (InternVL-2B) as the slow branch and construct a lightweight fast branch by replacing the VLM with a ViT backbone; both branches use the same diffusion-planner architecture but are trained as separate policies (no weight sharing). Since NAVSIM is one-shot (predicting a single 4s trajectory per scene), we use a simplified ASSCG without buffer/time features and with a binary decision (fast vs. slow). To better reflect the sensitivity to future trajectory quality in this setting, we additionally encode the fast-branch predicted trajectory with a lightweight MLP (mapped to a 256-d vector) and feed it to the gate as an auxiliary cue. The NAVSIM gate is trained with RL to directly optimize PDMS.

Table 2 shows a compact comparison. Our RecogDrive-based dual system with ASSCG achieves 91.4 PDMS, outperforming the original ReCogDrive (90.8) and reducing average wall-clock inference latency from ~ 350 ms to ~ 270 ms on a single A30 GPU. Although the system maintains two trained branches, only one

Table 2: Compact NAVSIM comparison. Full results are in Appendix E. [†]Concurrent preprints that appeared after our initial submission.

Method	NC↑	DAC↑	TTC↑	Comf.↑	EP↑	PDMS↑
UniAD [15]	97.8	91.9	92.9	100	78.8	83.4
TransFuser [7]	97.7	92.8	92.8	100	79.2	84.0
Hydra-MDP [21]	98.3	96.0	94.6	100	78.7	86.5
ReCogDrive [20]	97.9	97.3	94.9	100	87.3	90.8
DiffusionDriveV2 [44] [†]	98.3	97.9	94.8	99.9	88.0	91.2
<i>ours</i>	98.2	98.3	94.8	100	87.5	91.4

Table 3: Performance and per-frame inference latency.

Method	Score ↑	Latency (s/frame) ↓
AsyncDriver	65.00	0.80
AsyncDriver (5-frame interval)	64.27	0.32
AdaptiveAsyncDriver (ours)	67.28	0.32

branch is executed per scene after gating in this one-shot setting, so the reported wall-clock latency includes the gating overhead and selected branch inference. Full NAVSIM comparisons are provided in Appendix E.

6.4 Ablation Study

Table 4 reports ablations on training strategy and backbone choice for AdaptiveAsyncDriver.

Training strategy. Supervision-only gating achieves 66.21%. Adding on-policy fine-tuning with GRPO improves the score to 67.28%, indicating that, without access to optimal labels, post-SFT RL can outperform the fixed-interval surrogate by exploring and refining schedules, suppressing redundant calls in equivalent intervals, and skipping low-yield windows. Replacing GRPO with PPO yields 66.76%, but we observe instability and mode collapse, likely due to the value network’s difficulty in fitting the composite, rule-based score and estimating long-horizon returns.

Backbone choice. Replacing the RWKV backbone in ASSCG with a Transformer block of similar parameter count gives comparable accuracy (67.21% vs. 67.28%), but latency scales worse: Transformer attention is quadratic in context length L ($O(L^2)$ time/memory per layer), while RWKV scales linearly in time and $O(1)$ in memory per step after state caching. For $L = 150$ (covering a full scene), last-frame gate latency is ~ 0.06 s for Transformer vs. 0.02 s for RWKV on the same A30 GPU. We report *last-frame* gate latency as a practical worst-case proxy: in long-horizon episodes the gate runs with the maximum context length, which upper-bounds the additional per-step overhead and is more relevant for real-time responsiveness than average latency. Although windowing or

Table 4: Ablation study of AdaptiveAsyncDriver

AsyncDriver	SFT	PPO	GRPO	Score $\uparrow$
✓				65.00
✓	✓			66.21
✓	✓	✓		66.76
✓	✓		✓	67.28
Backbone	Score $\uparrow$		Last-frame latency(s) $\downarrow$	
RWKV block	67.28		0.02	
Transformer block	67.21		0.06	

truncation can reduce L , real driving often exhibits variable and long temporal contexts; RWKV’s linear scaling thus provides a more robust latency profile, making it our default backbone.

Additional ablations (e.g., query-cost weight λ_{call} , removing DROP, and DROP usage statistics), qualitative visualizations, and our RecogDrive-based NAVSIM instantiation are provided in the Appendix F. In particular, removing DROP lowers the score from 67.28 to 66.71 despite a similar query interval, showing that ASSCG does more than reduce query frequency: it can intentionally suppress harmful slow guidance.

7 Conclusion

In this work, we analyzed limitations of existing coordination schemes for fast–slow planning and introduced concepts such as the *Equivalent Interval* and *Failure Interval*. Building on this analysis, we proposed the Adaptive Slow-System Control Gate (ASSCG), an architecture-agnostic control interface that schedules and regulates interactions between a learning-based fast planner and an LLM-based slow system. We validated ASSCG on two representative fast–slow instantiations and benchmarks—AsyncDriver on nuPlan Hard20 closed-loop evaluation and a RecogDrive-based fast–slow planner on NAVSIM—showing improved performance–efficiency trade-offs (higher scores with fewer or better-timed slow-module calls). A limitation is that our experiments evaluate when to use a slow branch given existing fast–slow systems, rather than proving that LLM reasoning is necessary for every benchmark or scenario. Harder end-to-end long-tail benchmarks are a natural next step.

Acknowledgements

Funded by Xiongan AI Institute, Lenovo Research and Wuxi Research Institute of Applied Technologies, Tsinghua University under Grant 20242001120.

References

1. Chen, L., Zhang, Y., Ren, S., Zhao, H., Cai, Z., Wang, Y., Wang, P., Liu, T., Chang, B.: Towards end-to-end embodied decision making via multi-modal large language model: Explorations with gpt4-vision and beyond. *arXiv preprint arXiv:2310.02071* (2023)
2. Chen, L., Sinavski, O., Hünermann, J., Karnsund, A., Willmott, A.J., Birch, D., Maund, D., Shotton, J.: Driving with llms: Fusing object-level vector modality for explainable autonomous driving. In: *2024 IEEE International Conference on Robotics and Automation (ICRA)*. pp. 14093–14100. IEEE (2024)
3. Chen, Y., Ding, Z.h., Wang, Z., Wang, Y., Zhang, L., Liu, S.: Asynchronous large language model enhanced planner for autonomous driving. In: *European Conference on Computer Vision*. pp. 22–38. Springer (2024)
4. Chen, Y., Wang, Y., Zhang, Z.: Drivingsgpt: Unifying driving world modeling and planning with multi-modal autoregressive transformers. *arXiv preprint arXiv:2412.18607* (2024)
5. Cheng, J., Chen, Y., Chen, Q.: Pluto: Pushing the limit of imitation learning-based planning for autonomous driving. *arXiv preprint arXiv:2404.14327* (2024)
6. Cheng, J., Chen, Y., Mei, X., Yang, B., Li, B., Liu, M.: Rethinking imitation-based planners for autonomous driving. In: *2024 IEEE International Conference on Robotics and Automation (ICRA)*. pp. 14123–14130. IEEE (2024)
7. Chitta, K., Prakash, A., Jaeger, B., Yu, Z., Renz, K., Geiger, A.: Transfuser: Imitation with transformer-based sensor fusion for autonomous driving. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **45**(11), 12878–12895 (2022)
8. Cui, C., Ma, Y., Cao, X., Ye, W., Wang, Z.: Receive, reason, and react: Drive as you say, with large language models in autonomous vehicles. *IEEE Intelligent Transportation Systems Magazine* **16**(4), 81–94 (2024)
9. Dauner, D., Hallgarten, M., Geiger, A., Chitta, K.: Parting with misconceptions about learning-based vehicle motion planning. In: *Conference on Robot Learning*. pp. 1268–1281. PMLR (2023)
10. Fu, D., Li, X., Wen, L., Dou, M., Cai, P., Shi, B., Qiao, Y.: Drive like a human: Rethinking autonomous driving with large language models. In: *2024 IEEE/CVF Winter Conference on Applications of Computer Vision Workshops (WACVW)*. pp. 910–919. IEEE (2024)
11. H. Caesar, J. Kabzan, K.T.e.a.: Nuplan: A closed-loop ml-based planning benchmark for autonomous vehicles. In: *CVPR ADP3 workshop* (2021)
12. Hallgarten, M., Stoll, M., Zell, A.: From prediction to planning with goal conditioned lane graph traversals. In: *2023 IEEE 26th International Conference on Intelligent Transportation Systems (ITSC)*. pp. 951–958. IEEE (2023)
13. Han, W., Guo, D., Xu, C.Z., Shen, J.: Dme-driver: Integrating human decision logic and 3d scene perception in autonomous driving. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. vol. 39, pp. 3347–3355 (2025)
14. Hu, Y., Chai, S., Yang, Z., Qian, J., Li, K., Shao, W., Zhang, H., Xu, W., Liu, Q.: Solving motion planning tasks with a scalable generative model. In: *European Conference on Computer Vision*. pp. 386–404. Springer (2024)
15. Hu, Y., Yang, J., Chen, L., Li, K., Sima, C., Zhu, X., Chai, S., Du, S., Lin, T., Wang, W., et al.: Planning-oriented autonomous driving. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. pp. 17853–17862 (2023)

16. Huang, Z., Liu, H., Lv, C.: Gameformer: Game-theoretic modeling and learning of transformer-based interactive prediction and planning for autonomous driving. In: Proceedings of the IEEE/CVF International Conference on Computer Vision. pp. 3903–3913 (2023)
17. Li, J., Zhang, B., Jin, X., Deng, J., Zhu, X., Zhang, L.: Imagidrive: A unified imagination-and-planning framework for autonomous driving. arXiv preprint arXiv:2508.11428 (2025)
18. Li, Y., Shang, S., Liu, W., Zhan, B., Wang, H., Wang, Y., Chen, Y., Wang, X., An, Y., Tang, C., et al.: Drivevla-w0: World models amplify data scaling law in autonomous driving. arXiv preprint arXiv:2510.12796 (2025)
19. Li, Y., Wang, Y., Liu, Y., He, J., Fan, L., Zhang, Z.: End-to-end driving with online trajectory evaluation via bev world model. arXiv preprint arXiv:2504.01941 (2025)
20. Li, Y., Xiong, K., Guo, X., Li, F., Yan, S., Xu, G., Zhou, L., Chen, L., Sun, H., Wang, B., et al.: Recogdrive: A reinforced cognitive framework for end-to-end autonomous driving. arXiv preprint arXiv:2506.08052 (2025)
21. Li, Z., Li, K., Wang, S., Lan, S., Yu, Z., Ji, Y., Li, Z., Zhu, Z., Kautz, J., Wu, Z., et al.: Hydra-mdp: End-to-end multimodal planning with multi-target hydra-distillation. arXiv preprint arXiv:2406.06978 (2024)
22. Liao, B., Chen, S., Yin, H., Jiang, B., Wang, C., Yan, S., Zhang, X., Li, X., Zhang, Y., Zhang, Q., et al.: Diffusiondrive: Truncated diffusion model for end-to-end autonomous driving. arXiv preprint arXiv:2411.15139 (2024)
23. Peng, B., Alcaide, E., Anthony, Q., Albalak, A., Arcadinho, S., Biderman, S., Cao, H., Cheng, X., Chung, M., Grella, M., et al.: Rwkv: Reinventing rnns for the transformer era. arXiv preprint arXiv:2305.13048 (2023)
24. Qian, K., Ma, Z., He, Y., Luo, Z., Shi, T., Zhu, T., Li, J., Wang, J., Chen, Z., He, X., et al.: Fasionad: Fast and slow fusion thinking systems for human-like autonomous driving with adaptive feedback. arXiv preprint arXiv:2411.18013 (2024)
25. Scheel, O., Bergamini, L., Wolczyk, M., Osiński, B., Ondruska, P.: Urban driver: Learning to drive from real-world demonstrations using policy gradients. In: Conference on Robot Learning. pp. 718–728. PMLR (2022)
26. Shao, H., Hu, Y., Wang, L., Song, G., Waslander, S.L., Liu, Y., Li, H.: Lmdrive: Closed-loop end-to-end driving with large language models. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 15120–15130 (2024)
27. Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y., Wu, Y., et al.: Deepseekmath: Pushing the limits of mathematical reasoning in open language models. arXiv preprint arXiv:2402.03300 (2024)
28. Sharan, S., Pittaluga, F., Chandraker, M., et al.: Llm-assist: Enhancing closed-loop planning with language-based reasoning. arXiv preprint arXiv:2401.00125 (2023)
29. Tang, Z., Zhang, S., Deng, J., Wang, C., You, G., Huang, Y., Lin, X., Zhang, Y.: Vlmplanner: Integrating visual language models with motion planning. arXiv preprint arXiv:2507.20342 (2025)
30. Tian, X., Gu, J., Li, B., Liu, Y., Wang, Y., Zhao, Z., Zhan, K., Jia, P., Lang, X., Zhao, H.: Drivevlm: The convergence of autonomous driving and large vision-language models. arXiv preprint arXiv:2402.12289 (2024)
31. Treiber, M., Hennecke, A., Helbing, D.: Congested traffic states in empirical observations and microscopic simulations. *Physical review E* **62**(2), 1805 (2000)
32. Weng, X., Ivanovic, B., Wang, Y., Wang, Y., Pavone, M.: Para-drive: Parallelized architecture for real-time autonomous driving. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 15449–15458 (2024)

33. Wu, D., Han, W., Liu, Y., Wang, T., Xu, C.z., Zhang, X., Shen, J.: Language prompt for autonomous driving. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. vol. 39, pp. 8359–8367 (2025)
34. Xu, M., Cui, J., Cai, F., Shang, H., Zhu, Z., Luan, S., Xu, Y., Zhang, N., Li, Y., Cai, J., et al.: Wam-diff: A masked diffusion vla framework with moe and online reinforcement learning for autonomous driving. *arXiv preprint arXiv:2512.11872* (2025)
35. Xu, Z., Zhang, Y., Xie, E., Zhao, Z., Guo, Y., Wong, K.Y.K., Li, Z., Zhao, H.: Drivegpt4: Interpretable end-to-end autonomous driving via large language model. *IEEE Robotics and Automation Letters* (2024)
36. Yao, R., Wang, Y., Liu, H., Yang, R., Peng, Z., Zhu, L., Ma, J.: Calmm-drive: Confidence-aware autonomous driving with large multimodal model. *arXiv preprint arXiv:2412.04209* (2024)
37. Yuan, C., Zhang, Z., Sun, J., Sun, S., Huang, Z., Lee, C.D.W., Li, D., Han, Y., Wong, A., Tee, K.P., et al.: Drama: An efficient end-to-end motion planner for autonomous driving with mamba. *arXiv preprint arXiv:2408.03601* (2024)
38. Yuan, J., Sun, S., Omeiza, D., Zhao, B., Newman, P., Kunze, L., Gadd, M.: Rag-driver: Generalisable driving explanations with retrieval-augmented in-context learning in multi-modal large language model. *arXiv preprint arXiv:2402.10828* (2024)
39. Zhang, R., Xie, J., Zhang, W., Chen, W., Tan, X., Wan, X., Li, G.: Adadrive: Self-adaptive slow-fast system for language-grounded autonomous driving. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. pp. 5112–5121 (2025)
40. Zhang, Y., Zhang, S., Zhang, Y., Ji, J., Duan, Y., Huang, Y., Peng, J., Zahng, Y.: Multi-modality fusion perception and computing in autonomous driving. *J. Comput. Res. Dev* **57**, 1781–1799 (2020)
41. Zhang, Z., Li, X., Zou, S., Chi, G., Li, S., Qiu, X., Wang, G., Zheng, G., Wang, L., Zhao, H., et al.: Chameleon: Fast-slow neuro-symbolic lane topology extraction. *arXiv preprint arXiv:2503.07485* (2025)
42. Zheng, Y., Xing, Z., Zhang, Q., Jin, B., Li, P., Zheng, Y., Xia, Z., Zhan, K., Lang, X., Chen, Y., et al.: Planagent: A multi-modal large language agent for closed-loop vehicle motion planning. *arXiv preprint arXiv:2406.01587* (2024)
43. Zhou, Z., Cai, T., Zhao, S.Z., Zhang, Y., Huang, Z., Zhou, B., Ma, J.: Autovla: A vision-language-action model for end-to-end autonomous driving with adaptive reasoning and reinforcement fine-tuning. *arXiv preprint arXiv:2506.13757* (2025)
44. Zou, J., Chen, S., Liao, B., Zheng, Z., Song, Y., Zhang, L., Zhang, Q., Liu, W., Wang, X.: Diffusiondrivev2: Reinforcement learning-constrained truncated diffusion modeling in end-to-end autonomous driving. *arXiv preprint arXiv:2512.07745* (2025)

A Per-type fixed-schedule grid search on nuPlan Hard20

We provide detailed results of the fixed-schedule grid search mentioned in §3. For each scenario type (type0–type13), we evaluate a family of fixed query schedules parameterized by the query stride $K \in \{1, 3, 5, 10, 20, 50, 100\}$, where $K=1$ queries every frame. We also include *Never-Query*, which disables the slow module entirely. For each type, we report the best average score over its Hard20 episodes and the corresponding schedule (Table 5).

How to interpret the results. The wide variation in the best stride across types suggests that the usefulness of slow reasoning is highly scenario-dependent. Importantly, this grid search uses a *type-level* constant stride, i.e., the same schedule is applied to all episodes within a type. This restriction likely underestimates what an adaptive policy could achieve, because even within a fixed scenario type the need for slow guidance can change over time and differ across episodes (e.g., due to interaction density, occlusions, or traffic-light timing). Our goal is therefore to learn a *frame-level* gate that adapts query decisions to the evolving context rather than committing to a single hand-designed schedule.

Table 5: Per-type best fixed slow-module query schedule on nuPlan Hard20 (grid search). K is the fixed query stride; “Never” disables the slow module. Scenario types follow the official nuPlan taxonomy and correspond to type0–type13 in Table 9.

Scenario type	Best score	Best fixed schedule
Behind long vehicle	83.53	$K=1$
Changing lane	69.57	$K=10$
Following lane with lead	83.66	$K=5$
High lateral acceleration	63.02	Never
High magnitude speed	83.77	$K=3$
Low magnitude speed	50.00	$K=10$
Near multiple vehicles	6.27	$K=5$
Starting left turn	62.55	$K=10$
Starting right turn	66.12	$K=1$
Starting straight traffic lights traversal	70.99	$K=50$
Stationary in traffic	84.90	$K=10$
Stopping with lead	97.30	$K=10$
Traversing pickup/dropoff	55.37	$K=10$
Waiting for pedestrian	55.02	$K=20$

B Supplementary: RL Fine-tuning Objective and Details

SFT pseudo-label construction. The SFT labels are not assumed to be optimal frame-level decisions. They are an initialization derived from closed-loop rollouts: (i) for each training scene, we evaluate a fixed pool of query schedules with stride

$K \in \{1, 3, 5, 10, 20, 50, 100\}$ plus *Never-Query*; (ii) each rollout is scored by the official nuPlan closed-loop metric; (iii) the highest-scoring rollout for that scene is selected; and (iv) its per-frame QUERY/CACHE/DROP action sequence is used as the pseudo label. This logarithmic stride pool covers dense, medium, sparse, and no-query regimes while keeping simulator cost manageable. The subsequent RL stage is therefore responsible for refining beyond these coarse fixed schedules.

Logged data and behavior policy. We construct a dataset D of trajectories by rolling out the SFT-initialized policy. For each step we log $(s_t, a_t, r_t, \log b_t, \text{done})$, where $b(a_t|s_t)$ is the behavior policy used for data collection and $\log b_t = \log b(a_t|s_t)$ is stored for off-policy correction. The behavior policy is an ε -mixture of a temperature-scaled policy and the uniform distribution:

$$b(a|s) = (1 - \varepsilon(s)) \text{softmax}(\text{logits}(s)/T(s)) + \varepsilon(s) U(a). \quad (7)$$

Reward shaping. Per-step rewards are computed by potential-based shaping plus a query-cost penalty, as described in Eq. 5. The terminal step additionally receives the progress settlement term so that the shaped return matches the official score up to a policy-invariant constant.

Advantage estimation. We use generalized advantage estimation (GAE):

$$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t), \quad \hat{A}_t = \sum_{k \geq 0} (\gamma \lambda)^k \delta_{t+k}, \quad (8)$$

and the λ -return target $\hat{R}_t = \hat{A}_t + V_\phi(s_t)$ for value regression.

Off-policy correction and clipped policy objective. Let π_θ denote the current policy and π_{ref} the frozen SFT reference. We compute the importance ratio

$$\rho_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{b(a_t|s_t)} = \exp(\log \pi_\theta(a_t|s_t) - \log b_t), \quad (9)$$

and apply truncation $\tilde{\rho}_t = \min(\rho_t, \rho_{\max})$ to cap variance. The policy objective uses PPO-style clipping with KL regularization and entropy bonus:

$$\begin{aligned} \mathcal{L}_\pi(\theta) = \mathbb{E}_{(s_t, a_t) \sim D} \Big[& \min\left(\tilde{\rho}_t \hat{A}_t, \text{clip}(\tilde{\rho}_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t\right) \\ & - \beta \text{KL}(\pi_\theta(\cdot|s_t) \parallel \pi_{\text{ref}}(\cdot|s_t)) + \eta \mathcal{H}(\pi_\theta(\cdot|s_t)) \Big]. \end{aligned} \quad (10)$$

Value regression and overall optimization. We minimize mean squared error for the value function:

$$\mathcal{L}_V(\phi) = \mathbb{E}_{s_t \sim D} \left[(V_\phi(s_t) - \hat{R}_t)^2 \right], \quad (11)$$

and jointly optimize policy and value parameters by minimizing

$$\min_{\theta, \phi} -\mathcal{L}_\pi(\theta) + \lambda_V \mathcal{L}_V(\phi). \quad (12)$$

Hyperparameters. Unless otherwise stated: $\gamma=0.99$, $\lambda=0.95$, $\lambda_{\text{call}}=0.001$, $\epsilon=0.2$, $\rho_{\text{max}}=10$, $\beta=0.01$, $\eta=0.01$, and $\lambda_V=0.5$. We use AdamW with learning rate 10^{-4} and gradient-norm clipping at 1.0.

C nuplan Metrics

In this section, we elucidate the metrics of closed-loop evaluation deployed within the nuPlan [11] framework. The calculation for the composite metric *Score* is detailed in Formula 13.

Drivable Area Compliance (Drivable) Ego should remain within the designated drivable area. If a frame is identified where the maximum distance from any corner of the ego’s bounding box to the nearest drivable area exceeds a pre-determined threshold, the drivable area compliance score is assigned a value of 0; otherwise, it is assigned a value of 1.

Driving Direction Compliance (Direct.) The evaluation of the ego’s compliance with the correct driving direction is encapsulated within this metric. It stipulates that a distance traversed in the opposite direction of less than 2 meters within a 1-second interval is deemed compliant, warranting a score of 1. A traversal exceeding 6 meters within the same timeframe indicates a significant deviation from compliance, thus yielding a score of 0. Intermediate distances are assigned a proportional score of 0.5.

Ego is Comfortable (Comf.) This metric quantifies the comfort of the trajectory based on a series of kinematic and dynamic thresholds including longitudinal acceleration, lateral acceleration, yaw acceleration, yaw velocity, longitudinal jerk, and overall jerk magnitude. Compliance across all specified parameters results in a trajectory being classified as "comfortable", meriting a score of 1; otherwise, the score is 0.

Ego Progress Along Expert Route (Prog.) This metric represents the ratio of the ego’s progress per frame to that of an expertly defined route. The closer the ego’s progress is to the expert route’s progress, the higher the score, with the maximum score being 1.

No Ego At-fault Collisions (Coll.) This metric is integral to the safety evaluation, quantifying instances where the ego is directly responsible for collisions. An absence of at-fault collisions throughout the evaluation period scores a 1. A singular collision with a static object, such as a traffic cone, is deemed a minor infraction, scoring a 0.5, whereas all other scenarios result in a score of 0, accentuating the paramount importance of collision avoidance.

Speed Limit Compliance (Lim.) The ego’s adherence to speed regulations is scrutinized by calculating the average instance of speed limit violations per frame. This metric deducts points based on the proximity of the violation to the maximum speed threshold, promoting strict compliance with speed limits.

Time to Collision within Bound (TTC) This metric evaluates the hypothetical time to collision (TTC) with any external agent if the ego were to continue on its trajectory with unaltered speed and heading. A TTC exceeding 0.95 seconds is considered safe, thus earning a score of 1. Lower TTC values signify elevated

risk, consequently scoring a 0, highlighting the significance of maintaining a safe buffer from other road users.

Making Progress (MP) An ancillary metric to the Ego Progress Along Expert Route, it assigns a score of 1 if the progress exceeds a threshold of 0.2, otherwise scoring 0. Though not directly incorporated into the main text due to its derivative nature, it contributes to the final score.

Score The final scenario score is calculated by combining the above metrics in a predetermined manner, providing a comprehensive assessment of the autonomous vehicle’s performance within the evaluated scenarios:

$$\begin{aligned} \text{Score} = & (5 \times \text{Prog} + 5 \times \text{TTC} + 4 \times \text{Lim} + 2 \times \text{Comf}) \\ & \div 16 \times \text{Coll} \times \text{Drivable} \times \text{MP} \times \text{Direct} \end{aligned} \quad (13)$$

D Additional nuPlan Results

D.1 Per-scenario-type results on Hard20

Table 6 reports the per-scenario-type breakdown (type0–type13) on nuPlan Hard20. We include this table in the appendix for completeness and to facilitate comparison with prior work. The overall trend is consistent with the main text: ASSCG improves most scenario types while keeping the compute budget similar to a fixed-stride baseline, suggesting that the benefit comes from better timing and selective suppression of slow-module usage rather than higher query frequency.

Table 6: Per-scenario-type scores on nuPlan Hard20. Types 0–13 follow the official nuPlan taxonomy and correspond to the 14 scenario types. [†]VLM-planner does not report per-type scores; we denote missing entries as “–”.

Methods	score	type0	type1	type2	type3	type4	type5	type6	type7	type8	type9	type10	type11	type12	type13
UrbanDriver [25]	35.35	69.39	15.78	44.59	7.42	13.70	27.14	0.00	19.44	20.80	23.61	68.33	93.16	33.78	56.39
GCPGP [12]	36.85	59.50	35.91	33.60	29.33	42.84	17.24	4.86	32.33	8.22	36.23	71.32	80.46	36.19	23.61
IDM [31]	53.07	70.69	44.84	91.54	54.08	50.22	41.71	4.76	53.97	37.53	60.33	83.97	93.03	45.77	12.44
GameFormer [16]	62.05	84.32	65.78	83.62	49.03	71.79	36.80	0.00	51.76	55.03	77.24	82.83	<u>98.24</u>	49.41	56.16
PDM-Hybrid [9]	64.05	87.68	69.61	<u>87.20</u>	49.95	82.80	41.32	4.23	54.98	51.72	<u>82.95</u>	80.37	98.40	37.77	<u>71.99</u>
PDM-Closed [9]	64.18	<u>87.67</u>	<u>70.93</u>	<u>87.20</u>	49.95	82.80	41.25	4.22	53.53	51.57	82.96	80.37	98.40	39.95	72.04
VLM-planner [29] [†]	66.66	–	–	–	–	–	–	–	–	–	–	–	–	–	–
AsyncDriver [3]	65.00	83.53	67.24	83.14	<u>62.89</u>	<u>83.28</u>	<u>49.26</u>	<u>5.94</u>	62.53	<u>65.96</u>	64.86	<u>84.88</u>	96.62	<u>49.60</u>	54.35
ours	67.28	83.40	72.00	84.50	63.25	84.11	50.02	6.15	<u>62.45</u>	66.02	73.00	84.91	97.24	60.07	55.00

D.2 Broader validation and control policies

On the larger Val14 split, ASSCG also improves over the all-query AsyncDriver baseline (77.78 vs. 75.41), suggesting that the learned gate is not specific to the small Hard20 subset. Table 7 compares ASSCG with simple control policies on

Table 7: Additional control policies on nuPlan Hard20. Random scores are averaged over three seeds.

Policy	Score $\uparrow$	Avg. query interval
All QUERY (AsyncDriver)	65.00	1.00
All DROP (GameFormer)	62.05	–
First QUERY, then always CACHE	63.71	149.0
Random uniform actions	62.08	–
Random matched action ratio	62.79	–
ASSCG	67.28	5.26

Hard20. The first frame is always forced to QUERY, so the invalid case of selecting CACHE before any buffer is available has zero occurrence in all experiments.

The learned action ratio is approximately $\text{CACHE:QUERY:DROP} \approx 6 : 2 : 1$, and the average query intervals across the 14 scenario types are 3.3, 3.1, 15.0, 6.6, 29.3, 3.2, 9.2, 4.5, 7.9, 3.3, 26.8, 4.2, 4.4, and 4.7 frames. This spread supports the motivation for frame-level adaptive control rather than a single global fixed stride.

Runtime and delayed-query robustness. AsyncDriver is asynchronous: the fast planner continues to produce controls while slow guidance is being refreshed. The remaining runtime bottleneck is therefore the slow branch, not ASSCG itself. In our setting, the theoretical worst case (slow query plus gate overhead) is about 0.82s, while the slowest scene averages about 0.44s/frame. To probe delayed slow responses, we delay every QUERY by 3 frames, corresponding to the maximal offset under an approximate 3:1 slow/fast runtime ratio. The score drops mildly from 67.28 to 66.76, still above the all-query AsyncDriver baseline.

E Additional Results on NAVSIM

E.1 Benchmark overview and metric

We additionally evaluate on NAVSIM, a planning-oriented benchmark with a non-reactive (pseudo closed-loop) simulator: the ego policy does not affect other agents, and each test case requires predicting a single future trajectory for a short horizon. Following NAVSIM, we report the official Predictive Driver Model Score (PDMS), where higher is better:

$$\text{PDMS} = \text{NC} \times \text{DAC} \times \left(\frac{5 \cdot \text{EP} + 5 \cdot \text{TTC} + 2 \cdot \text{C}}{12} \right), \quad (14)$$

where NC denotes no at-fault collisions, DAC drivable-area compliance, EP ego progress, TTC time-to-collision within bound, and C comfort.

E.2 RecogDrive-based fast-slow instantiation

ReCogDrive [20] combines a vision-language model (InternVL-2B) with a diffusion-based planner. To construct a fast-slow system, we keep the original InternVL-2B branch as the *slow* module and build a *fast* branch by replacing the VLM with a lightweight vision-only backbone (ViT-large). Both branches use the same downstream diffusion-planner architecture but are trained as separate policies (no weight sharing). We then apply ASSCG to select between fast and slow outputs.

E.3 ASSCG adaptation for one-shot prediction

Unlike nuPlan closed-loop planning, NAVSIM evaluates one-shot 4s trajectory prediction (8 waypoints at 0.5s). Therefore, we use a simplified variant of ASSCG: we remove the buffer and time-since-last-call features and collapse the action space to a binary gate (select FAST vs. SLOW). Since the decision is one-shot, we train the gate with RL to directly optimize the PDMS score (without potential-based shaping).

E.4 Results

Table 9 reports NAVSIM performance. In addition to PDMS, we report wall-clock inference latency per scene (measured on a single NVIDIA A30). Our RecogDrive-based dual system with ASSCG improves PDMS while reducing average inference latency from ~ 350 ms to ~ 270 ms.

Deployment cost and slow-branch usage. The NAVSIM instantiation stores two trained branches, but after the binary gate only one branch is executed for a scene. On a single A30, the standalone fast branch takes about 97 ms, the slow ReCogDrive branch about 355 ms, and the gate about 19 ms. The learned policy selects the slow branch for about 40% of scenes; the reported ~ 270 ms average is measured end-to-end and includes the gate, selected branch inference, and common pre/post-processing overhead. Replacing the InternVL-based slow branch with a separately tuned Qwen3VL-8B slow branch gives 91.37 PDMS with a $\sim 24.7\%$ speedup over always using that slow branch, further suggesting that the gating principle is not tied to a single slow model.

E.5 NavHard results

We also evaluate the RecogDrive-based instantiation on NavHard, the harder NAVSIM subset suggested by reviewers. Without additional backbone fine-tuning, ASSCG improves over ReCogDrive and the ViT-only fast branch (Table 8). The absolute scores remain low for all compared VLA/VLM-style methods, especially in stage 2, which indicates that NavHard is a challenging benchmark and that stronger slow-branch modeling is complementary to our gating objective.

Table 8: NavHard results. Stage-1 and stage-2 scores follow the NavHard protocol.

Method	Overall $\uparrow$	Stage-1 $\uparrow$	Stage-2 $\uparrow$
ReCogDrive	26.2	68.9	37.7
ReCogDrive-ViT (fast only)	27.1	67.2	40.1
DiffusionDrive	27.0	66.7	40.5
Curious-VLA	27.6	—	—
ReCogDrive-ASSCG	28.0	68.7	40.2

Table 9: NAVSIM results on *navtest* using PDMS.[†]Concurrent preprints that appeared after our initial submission.

Method	NC $\uparrow$ DAC $\uparrow$		TTC $\uparrow$ Comf. $\uparrow$		EP $\uparrow$	PDMS $\uparrow$
DrivingGPT [4]	98.9	90.7	94.9	95.6	79.7	82.4
UniAD [15]	97.8	91.9	92.9	100	78.8	83.4
TransFuser [7]	97.7	92.8	92.8	100	79.2	84.0
PARA-Drive [32]	97.9	92.4	93.0	99.8	79.3	84.0
DRAMA [37]	98.0	93.1	94.8	100	80.1	85.5
Hydra-MDP [21]	98.3	96.0	94.6	100	78.7	86.5
ImagiDrive [17]	98.1	96.2	94.5	100	80.5	86.9
DiffusionDrive [22]	98.2	96.2	94.7	100	82.2	88.1
WoTE [19]	98.5	96.8	94.9	99.9	81.9	88.3
AutoVLA [18]	98.4	95.6	98.0	99.9	81.9	89.1
DriveVLA-W0 [43]	98.7	99.1	95.3	99.3	83.3	90.2
ReCogDrive [20]	97.9	97.3	94.9	100	87.3	90.8
WAM-diff [34] [†]	99.1	98.3	96.5	99.9	84.4	91.0
DiffusionDriveV2 [44] [†]	98.3	97.9	94.8	99.9	88.0	91.2
<i>ours</i>	98.2	98.3	94.8	100	87.5	91.4

F Additional Ablations and Qualitative Results

F.1 Effect of query-cost weight

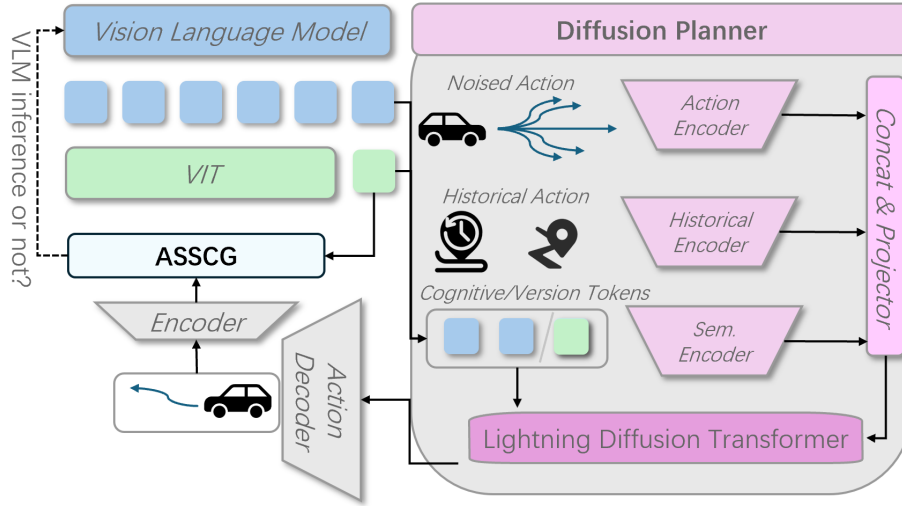
We study the impact of the query-cost weight λ_{call} (Eq. 5) on the performance–efficiency trade-off. Table 10 reports the overall Hard20 score and the average effective query interval under different λ_{call} values.

F.2 Confidence feature ablation

AdaDrive-style activation mechanisms use scene complexity and confidence to modulate LLM assistance. In our AsyncDriver-based setting, adding a confidence cue to ASSCG changes the score only marginally (67.28 to 67.33) while shortening the average query interval from 5.26 to 4.95 frames. We therefore keep the default gate focused on temporal context, buffer similarity, and elapsed time since the last slow query; confidence may be more useful in architectures

Table 10: Ablation on λ_{call} (nuPlan Hard20).

λ_{call}	Score $\uparrow$	Avg. query interval (frames) $\uparrow$
0.0005	67.25	1.91
0.0010	67.28	5.26
0.0015	66.90	5.69
0.0020	65.61	8.28

**Fig. 4:** RecogDrive-based fast-slow system for NAVSIM. The vision-only fast branch and VLM-based slow branch produce candidate trajectories using the same diffusion-planner architecture; a simplified ASSCG (binary gate) selects the output.

where candidate-trajectory confidence is less entangled with the fast planner’s own aggregation module.

F.3 RecogDrive-based fast-slow instantiation on NAVSIM

Fig. 4 illustrates our RecogDrive-based fast-slow instantiation used on NAVSIM. The slow branch follows the original ReCogDrive design with a VLM (InternVL-2B), while the fast branch replaces the VLM with a lightweight vision-only backbone (ViT-large). Both branches use the same downstream diffusion-planner architecture but are trained as separate policies (no weight sharing). A binary gate selects between the two branches for one-shot trajectory prediction.

F.4 Additional qualitative visualizations

We provide two additional qualitative comparisons on nuPlan Hard20 (Fig. 5). Each column corresponds to one scenario (left/right). For each scenario, we

Table 11: Effect of removing DROP on nuPlan Hard20.

Variant	Score $\uparrow$	Avg. query interval (frames) $\uparrow$
ASSCG (Query/Cache/Drop)	67.28	5.26
ASSCG w/o Drop (Query/Cache)	66.71	5.44

Table 12: DROP usage statistics on nuPlan Hard20.

Statistic	Value	Notes
Drop ratio (frames)	12	% of steps with $a_t = \text{DROP}$
Episodes with Drop	45	% of episodes containing at least one DROP

visualize 7 synchronized snapshots; the first row shows AsyncDriver (always querying the slow module) and the second row shows our method with ASSCG.

Scenario A (left). The 7 frames correspond to steps $\{0, 10, 30, 50, 70, 90, 100\}$. During the first half of the episode (up to ~ 50), ASSCG issues only three slow-module queries while producing behavior nearly indistinguishable from per-frame querying, illustrating effective reuse within an equivalent interval. Around step ~ 60 , ASSCG outputs DROP to suppress the slow guidance; the ego vehicle decelerates and avoids a subsequent collision.

Scenario B (right). The 7 frames correspond to steps $\{0, 10, 30, 50, 70, 90, 110\}$. ASSCG triggers the slow module only 6 times over the entire episode. Around step ~ 90 , ASSCG outputs DROP and the ego commits to a closer-to-stop behavior, preventing a collision with the vehicle on the left.

F.5 Ablation on the DROP action

Removing DROP. We ablate the DROP action by restricting ASSCG to a 2-way gate (QUERY/CACHE) while keeping all other settings unchanged. As shown in Table 11, removing DROP reduces the overall score ($67.28 \rightarrow 66.71$) despite a similar average query interval, indicating that selectively suppressing slow guidance is important beyond merely controlling query frequency.

DROP usage statistics. Table 12 reports how frequently DROP is used. We report (i) the fraction of frames where ASSCG outputs DROP and (ii) the fraction of episodes that contain at least one DROP decision.

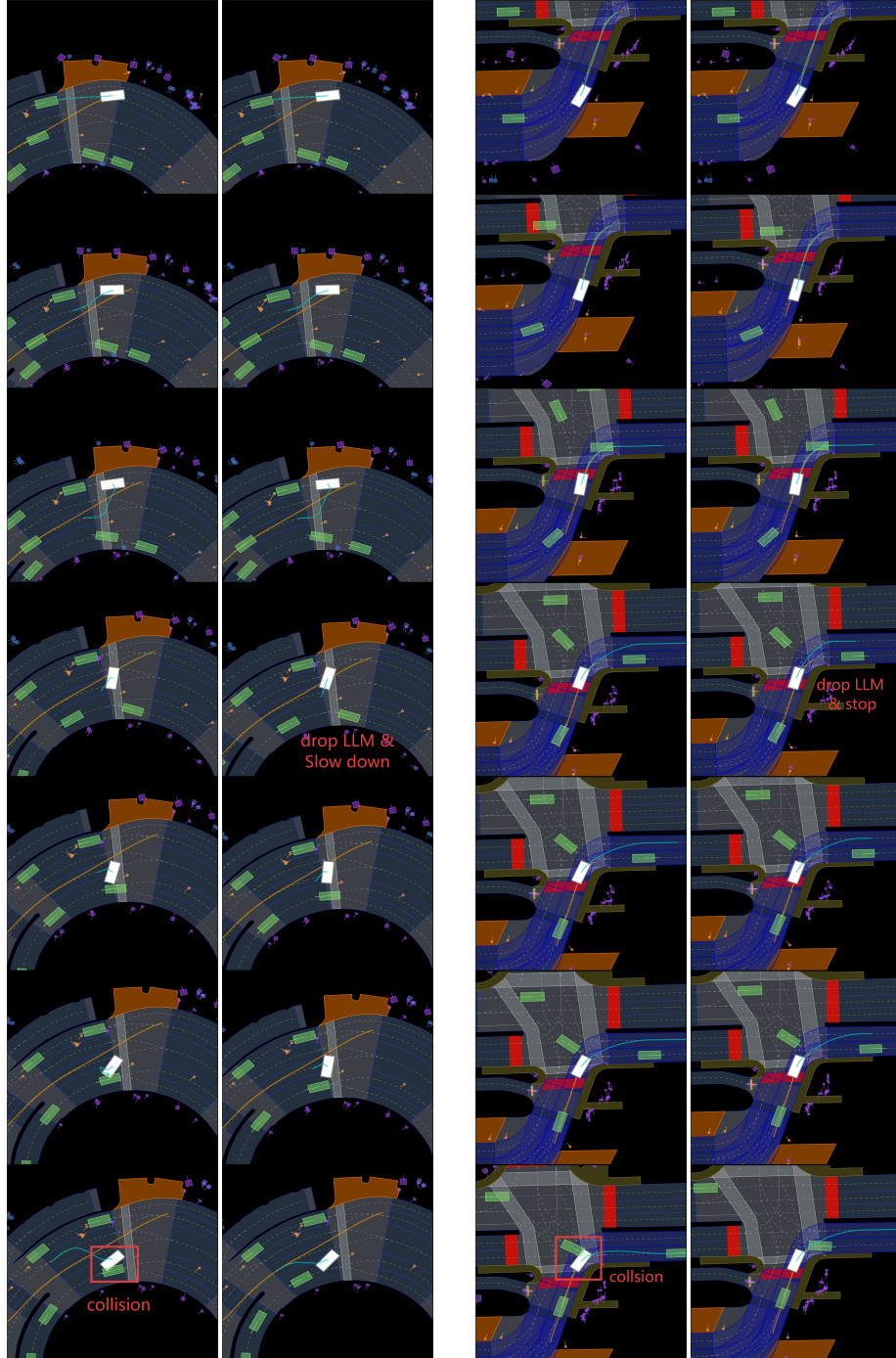


Fig. 5: Additional qualitative examples on nuPlan Hard20.